Multi-Agent Specification-based Metamorphic Testing of FMU-Based Simulations

Ashir Kulshreshtha*, Abdullah Mughees*, Gaadha Sudheerbabu*, Tanwir Ahmad*, Kristian Klemets†
Dragos Truscan*, and Mikael Manngård‡,
*Åbo Akademi University, Finland (firstname.lastname@abo.fi)
†University of Turku, Finland (firstname.lastname@utu.fi)
‡Novia University of Applied Sciences, Finland (firstname.lastname@novia.fi)

2026 IEEE 50th Annual Computers, Software, and Applications Conference (COMPSAC) | 979-8-3315-4497-3/26/$31.00 ©2026 IEEE | DOI: 10.1109/COMPSAC69091.2026.00424

***Abstract*—In many industrial domains, the Functional Mock-up Interface (FMI) standard is used to exchange simulation models as Functional Mock-up Units (FMUs) across different partners using various modelling tools. This opens up the possibilities for simulation-based verification and validation using FMUs for ensuring reliable system behaviour. However, deriving effective test oracles for these simulation models remains challenging due to the absence of explicit expected outputs. This limits the applicability of conventional testing approaches, which require access to the internal workings of the systems. Metamorphic testing (MT) addresses this limitation by leveraging metamorphic relations (MRs), but extracting such relations from specifications remains largely a manual and error-prone process. To address this challenge, we propose an LLM-powered multi-agent workflow for specification-based metamorphic testing of FMU-based simulation models. The approach takes functional and interface specifications as input and orchestrates multiple agents to extract requirements and derive MRs. These MRs are expressed using Given–When–Then (GWT) requirement patterns to structure input conditions (Given), transformations (When), and expected output behaviours (Then). These relations are then used to generate metamorphic test cases, execute simulations, and evaluate output consistency across multiple sessions. We evaluate the approach on a Lube Oil Cooling system FMU, demonstrating its ability to automatically generate meaningful MRs and corresponding test cases. Preliminary results indicate that the proposed workflow can support systematic verification and validation (V&V) of dynamic simulation models by reducing manual effort in MR identification and test generation.**

***Index Terms*—Metamorphic testing, multi-agent systems, AI-based testing, Large language models**

I. Introduction

In modern industrial systems, model-based design is an increasingly integral approach to system development, in which components are designed and validated through simulation prior to physical implementation [1]. Organizations typically use specialized modelling and simulation tools for model-based development and often collaborate with internal and external partners in this process. Development strategy using component-based design in such collaborative settings necessitates a standardized way of model exchange and co-simulation.

The Functional Mock-up Interface (FMI) [2] is a standard for tool-independent exchange of dynamic simulation models. It defines a container and an interface to exchange simulation models as Functional Mock-up Units (FMUs). An FMU is a ZIP archive that typically contains: (i) binaries and/or source to execute the model, (ii) model description, and (iii) optional resources (such as documentation files, tables needed by the model, and/or all object libraries or DLLs that are utilized).

This encapsulation brings along several challenges for V&V of the FMU components: a) often the source code of the model and its internal functionalities are not included in the package with it, allowing only for specification-based testing techniques to be applied, and b) the test oracles are not explicitly defined since typically the testing of such simulation models is done manually by domain experts. However, the simulation model of a component/system is designed based on the specifications of the real system it represents and is available for validation early in the development. This opens up the possibility of early simulation-based testing using specification-based testing techniques.

Metamorphic testing (MT) is a specification-based testing technique that can be used to test systems lacking explicit test oracles [3]. MT checks whether multiple executions of the system under test (SUT) satisfy specific necessary properties, called *metamorphic relations* (MRs). It starts with a *seed input* and derives one or more *follow-up inputs* by applying a *metamorphic transformation*. Instead of checking one output against a fixed expected value, the test verdict is assigned based on whether the MR that links the seed and follow-up outcomes holds [4]. MT has been applied in many application domains, including simulation and modelling [5]–[9].

Despite prior studies on generating MRs and MT for simulation models, there is still no FMU-focused approach that starts from functional and interface specifications and continues through MR generation, test generation, simulation execution, and test-quality assessment. Moreover, identifying suitable MRs remains a practical challenge when exploring the applicability of MT to new application domains [10], [11]. FMU-based simulations introduce practical constraints, for example, the internal model implementation may be unavailable, the executable interface is limited to exposed FMU variables, and not every requirement in the functional specification can be directly converted into a valid input-output MR. These constraints motivate the need for an automated workflow that identifies MRs from specifications, then designs, executes metamorphic tests, and reports requirement coverage as a test-adequacy indicator and output-level mutation score as a test-quality indicator.

Large language models (LLMs) are increasingly used for

2836-3795/26/$31.00 ©2026 IEEE
DOI 10.1109/COMPSAC69091.2026.00424

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on September 08,2026 at 00:57:36 UTC from IEEE Xplore. Restrictions apply.

automated test generation, and recent studies [12]–[14] show that LLMs can identify MRs in different application domains, such as web applications, autonomous driving systems, and embedded systems. Although these studies demonstrate the effectiveness of LLMs in generating innovative and diverse MRs, they also emphasize the need for a domain expert to validate the MRs to ensure high accuracy. Open challenges remain in systematically identifying effective MRs and reducing reliance on domain experts [11].

In this paper, we investigate the following research questions:

- *RQ1:* Can a multi-agent workflow systematically perform automated MT from the functional and interface specification of an FMU?
- *RQ2:* To what extent does the approach extract specification-linked MRs and executable metamorphic test cases based on the FMU specification?
- *RQ3:* What is the quality of the generated test cases?
- *RQ4:* What are the main runtime costs of the workflow across extraction, MR generation/refinement, test generation, and test execution?

To answer these research questions, we introduce **AgenticMeta**, an LLM-powered multi-agent workflow to support MT of FMU-based simulation models using the functional and interface specifications as the primary inputs. The key contributions of this paper are as follows:

- An extensible multi-agent approach for FMU-based metamorphic testing that extracts requirements from functional and interface specifications and then identifies MRs using LLM-powered agents orchestrated by a central control node.
- The extracted MRs are specified as Given-When-Then (GWT) [15] requirement patterns over time series, allowing for better processing by LLMs to generate metamorphic tests with customized input transformations and output relations for simulation models.
- A feasibility evaluation on a simplified version of the Lubricating Oil Cooling system FMU across multiple independent sessions, reporting requirement coverage, test execution outcomes, runtime statistics, and output-level mutation analysis.

The rest of this paper is organized as follows. Section II discusses related work. Section III presents the AgenticMeta workflow and its main components. Section IV describes the experimental design and reports the evaluation results. Section V discusses the limitations. Section VI concludes the paper and outlines future work.

II. Related Work

A. LLM-assisted MR Generation

The application of LLMs for automating different phases of MT has been investigated. Several studies exploring the applicability of LLMs for automatically generating MRs demonstrated their usability [12]–[14]. In [12], a zero-shot prompting strategy is used to generate MRs, and these MRs are verified by domain experts. Their study, conducted across nine software systems, suggested that ChatGPT-generated MRs can be used to generate innovative MRs and highlighted the need for human-in-the-loop validation. ChatGPT is used to generate MRs for autonomous driving systems [13], and it is reported that the approach can generate diverse MRs that effectively improve coverage using a feedback-based prompting strategy. The third study [14] uses a few-shot prompting strategy to derive MRs from natural-language requirements and convert them into executable forms using a domain-specific language called SMRL. Their work shows that LLMs can help identify transformation-based test relations and reduce manual effort.

B. LLM-assisted Metamorphic Testing

LLM-assisted MT has also been applied to different domains like embedded systems and the simulation and modelling domains. A recent related work proposed a multi-agent LLM-based MT approach for REST APIs, through a multi-agent framework. Even though it is similar to ours in generating tests from structured interface specifications, it addresses a different application domain, namely testing of dynamic simulation models [16]. More recent work [17] proposes an LLM-based workflow for MT integrated with the Gotten framework for MR inference and follow-up test generation. Their approach includes syntactic and semantic validation and fixing cycles to improve the precision of inferred MRs and follow-up tests. Contrary to their method, which primarily supports testers with an LLM-based assistive workflow for selected MT tasks within a general MT framework, our work targets a complete workflow for MT of FMUs, from requirement extraction to test results. Another recent work [18] presents a multi-agent tool that applies MT to autonomous driving systems. It derives MRs from driving rules, creates follow-up tests from images, executes them, and reports violations. In contrast, our approach targets dynamic simulation models, where tests are continuous time-series signals, and focuses on inferring, structuring, and refining MRs from functional specifications.

Compared with prior MT work for simulation models, AgenticMeta focuses specifically on FMU-based simulations where the testable interface is constrained by the variables exposed in the FMU model description. Earlier GWT-based MT work for dynamic simulation models [9] provides a useful structure for expressing MRs; this paper extends that work by automating requirement extraction, MR generation, MR refinement, test generation, and validation through LLM-based agents. Recent LLM-assisted MR-generation studies mainly focus on producing candidate MRs, whereas AgenticMeta automates the entire MT process, integrating MR refinement, test execution, and output-level mutation analysis into a single process. Compared with multi-agent MT for autonomous driving systems, our target domain is FMU-based dynamic simulation, where inputs and outputs are continuous time-series variables and where valid MRs must conform to the FMU interface constraints. Therefore, the primary contribution of this work resides not in the application of MT or LLMs in isolation, but in the integration of these components into a traceable, FMU-oriented MT workflow.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on September 08,2026 at 00:57:36 UTC from IEEE Xplore. Restrictions apply.

III. Overview of the Approach

A. Conceptual approach

Our approach aims to augment the main steps of the MT process, namely, systematic identification and selection of MRs from functional and interface specifications, and metamorphic test generation in patterns similar to Gherkin's language GWT requirement patterns [15]. As such, it defines the initial input context, the applied transformations, and the expected output relation with an LLM-powered multi-agent workflow for validation of dynamic simulation models.

The workflow comprises four phases: extraction, MR generation, test generation, and test execution, as illustrated in Figure 1. The first three phases are performed by specialized agents, whereas the last is performed by deterministic custom tools.

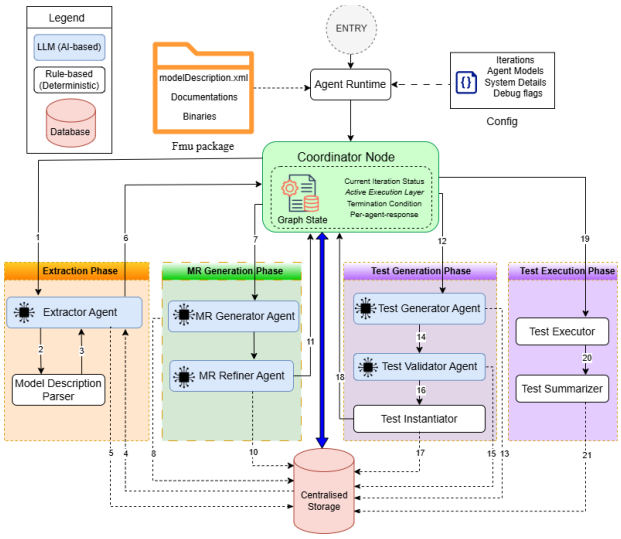


Fig. 1: Overview of the Multi-Agent workflow

In the *Extraction Phase*, we extract system properties to be selected for MR selection from the functional and interface specifications of the simulation model. The system properties specified in the functional specification are extracted as *test conditions*. In addition to this, the cause-and-effect relationship between the input and output variables in a specified property is extracted from the specification document. From the model description XML file in the FMU package, the model variables and the interface specification, including inputs, outputs, parameters, and their details (name, description, variability, unit, data type, minimum, and maximum values), are extracted. The extraction of relevant information and storing it in a structured manner as specified above streamlines the identification and selection of the MRs in the next phases.

Subsequently, in the *MR Generation Phase*, MRs are generated with an MR identifier and are mapped to the test condition identifier it corresponds to, along with the test scenario description. The generated MR has the metamorphic source and follow-up test specified in the GWT pattern with *Given*, *When*, and *Then* keywords. The *Given* states the initial conditions to generate seed inputs for the source test execution, *When* states the metamorphic input relation for transforming the seed inputs to morphed inputs, and *Then* states the test oracle as a metamorphic output relation between the outputs generated using the seed inputs and follow-up inputs. In the *Test Generation Phase*, the test design specified in each MR is converted into seed and follow-up input signals as time series, each over a specified time interval for each SUT input. In the *Test Execution Phase*, which is the last step in the workflow, we run and evaluate the generated time series against the FMU.

B. Multi-agent workflow

The agents, central to our workflow, perform well-defined tasks in a coordinated manner to facilitate LLM-assisted MT. The approach follows a 'hub and spoke' architecture in which the Coordinator node acts as the hub, monitoring and orchestrating the workflow and functioning of the agents and modules operating as spokes, with clearly defined roles. The approach is iterative and incremental.

The inputs to the workflow are: the FMU file of the model under test, including the model description, functional specification as a PDF, binaries of the model, and a set of configuration parameters for the workflow. The following parameters can be used to customize the workflow for a given case study:

- *system_name*: Name of SUT.
- *system_abv*: Short abbreviation for the SUT.
- *fmu_path*: Path to the FMU file used for simulation.
- *output_dir*: Directory name where run outputs are saved.
- *max_iterations*: Max number of iterations per run.
- *mr_count*: Max number of MRs to generate per iteration.
- *test_cases_per_mr*: Max number of test cases per MR.
- *llm_provider*: LLM service provider identifier.

We detail the role of each component of the workflow in the following.

1) Coordinator Node: maintains the global execution state, determines the active phase of the workflow, and routes the flow to the corresponding node. This design separates control-flow management from task-specific execution, enabling each LLM agent or deterministic module to operate on a well-defined responsibility, while the Coordinator governs the overall progression of the pipeline. A shared graph state is a central context that manages execution information, tracks intermediate states, and propagates coordination signals across nodes throughout the workflow.

At each workflow step, the Coordinator inspects the *phase* field in the shared graph state and applies a phase-to-node mapping to select the next executable component. After an iteration is completed, it records execution-related statistics. If the configured maximum number of iterations has not yet been reached, the Coordinator prepares for the next iteration by initialising iteration-specific directories and resetting iteration local artifacts to their initial state. When the iteration count is exhausted, the Coordinator marks the workflow as completed and exits the execution according to the configured condition.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on September 08,2026 at 00:57:36 UTC from IEEE Xplore. Restrictions apply.

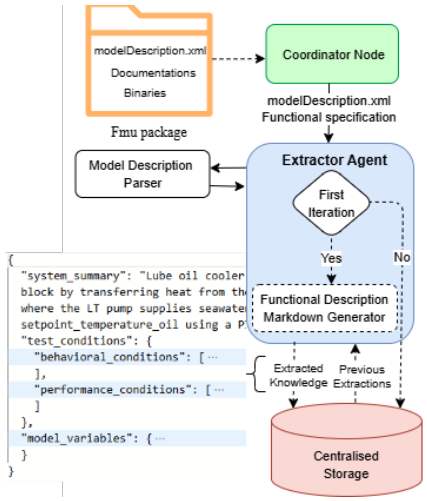


Fig. 2: Workflow of Extractor Agent

2) Extractor Agent: The *Extractor Agent* operates by transforming the input functional specification document through a structured conversion pipeline, then formulating an output in an LLM-friendly content format (see Figure 2). It transforms the functional specification document from PDF to Markdown, a structured format that enhances the processing and accuracy of agents that accept it as input for proceeding with further steps of MT. The system prompt to define the role of the agent for performing the information extraction is shown in Listing I.

System Prompt

Role:
You are a domain expert in Functional Mockup Interface (FMI) and Functional Mockup Unit (FMU) systems, with strong experience in requirement extraction and structured evidence preservation.
Core Principles:
1. Preserve document context
2. Accept equivalent table layouts (grid/pipe/html/text) as having the same semantic structure.
3. Document is authoritative - prioritize document facts as the single source of truth.
4. Verbatim evidence - every extracted fact (relationship, formula, constraint, test condition) must be grounded in a direct quote or precise reference from the document.
5. Tables are primary sources - treat input/output tables, parameter tables, and model description tables as the authoritative metadata source for variable specifications.
6. Precision over inference
7. Preserve relationships
8. Valid JSON output only - return well-formed JSON without markdown fences, commentary, or embedded natural language.

Listing I: System prompt of Extractor Agent

The model interface data and the constraints for input and output variables, including names, units, causality, and allowed ranges, are extracted from the `modelDescription.xml` file of the FMU. A custom XML parser script extracts and converts the information to JSON format.

To summarize, the *Extractor Agent* generates extraction output in a structured schema as defined in the task prompt. The output in nested JSON format comprises the following information to facilitate the MR generation: system summary, test conditions (categorized by property-type if relevant details are present in the functional specification), and model variables.

3) MR Generator Agent: In each iteration, the *MR Generator Agent* proposes a number of MRs as specified by the *mr_count* workflow configuration parameter. The task prompt designed for the *MR Generator Agent* guides the generation of candidate MRs. An excerpt of the task prompt, which constrains the generated MRs to follow the schema and to assign priorities as per the rules, is shown in Listing II.

Task Prompt

Task:
- Generate up to mr_count NEW, UNIQUE, CONCRETE and test conditions focused Metamorphic Relations (MRs).
- Use concise test conditions focused **Given-When-Then** form.
- Each MR MUST include: id, req_ids, scenario, given, when, then.
- Prevent duplicates with current & previous MRs.
- Keep the wording compact and clear in given, when and then.
Prioritise MR generation categories in this order:
priority
Constraint by Category:
- Behavioral
- Performance

Listing II: Task Prompt of MR Generator Agent

The prompt explicitly requires the candidates to be different from those generated in previous iterations by providing the agent with the accumulated MR history of up to three previous generations. Each candidate MR is assigned a unique MR identifier and linked to the requirements it addresses by listing the test condition identifier and variable relationship identifier.

The agent generates MRs as GWT patterns under a strict JSON schema using the rules defined in the task prompt, so that subsequent stages can reliably generate metamorphic tests. Figure 3 shows an example MR generated by the agent in the structured schema as per the rules defined in the task prompt in Listing II. In the example, *MR001*, the initial condition values for the seed input signals are stated in the *Given* part. The metamorphic transformation in the *When* part of the inferred MR states that the input signal `engine_load` is transformed from its seed value to a morphed value using the relational operator *'increase'*. The agent generates transformations aligned with rules specified in its prompt for the *When* using patterns such as *'STEP'*, *'RAMP'*, while respecting the constraints of the FMUs.

The metamorphic output relation to determine the test outcome is stated in the *Then* part using comparative and temporal operators such as the ones listed below:

- *Eventually_Increases than*: Implies that morphed outputs of the selected output variable should eventually exceed the corresponding seed outputs.
- *Eventually_Decreases than*: Implies that morphed outputs of the selected output variable should eventually decrease more than the corresponding seed outputs.
- *Proportional_to*: Implies that morphed outputs of the selected output variable should always be proportional to the corresponding seed outputs.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on September 08,2026 at 00:57:36 UTC from IEEE Xplore. Restrictions apply.

```
{
"id": "MR001",
"req_ids": [
    "BC006",
    "BC007"
],
"category": "Behavioral",
"scenario": "Rising engine_load effect on oil and cooling-liquid outlet temperatures.",
"given": "engine_load, temperature_cooling_liquid_in, and mass_flow_cooling_liquid_in
are at seed values; setpoint_temperature_oil is constant.",
"when": "Increase engine_load from the seed value while keeping
temperature_cooling_liquid_in, mass_flow_cooling_liquid_in, and
setpoint_temperature_oil unchanged.",
"then": [
    "temperature_oil eventually_increasing",
    "temperature_cooling_liquid_out eventually_increasing"
]
}
```


Fig. 3: MR generated by the agent in Given-When-Then pattern


```
{
"id": "MR001",
"req_ids": [
    "BC006",
    "BC007"
],
"category": "Behavioral",
"scenario": "Rising engine load effect on oil and cooling-liquid outlet temperatures.",
"given": "The system is at steady state, and temperature_cooling_liquid_in,
mass_flow_cooling_liquid_in, and setpoint_temperature_oil are held constant.",
"when": "Increase engine_load while keeping temperature_cooling_liquid_in,
mass_flow_cooling_liquid_in, and setpoint_temperature_oil unchanged.",
"then": [
    "temperature_oil eventually_increasing",
    "temperature_cooling_liquid_out eventually_increasing"
],
"dropped": false,
"feedback": "Supported by BC006 and BC007; removed ambiguous seed-value wording and
kept the perturbation focused on engine_load."
}
```


Fig. 4: MR refined by the agent and the feedback provided

- *Equal_to*: Implies that the morphed output value of the selected variable should always be equal to the corresponding seed output value.
- *Settles_within*: Implies that seed and morphed outputs of the selected output variable should settle within X seconds to a defined set_point value.

In total, the agent generates a maximum of $mr_count * iteration_count$ number of candidate MRs, where mr_count is defined by *MR Generator Agent* and $iteration_count$ by the *Coordinator node* for metamorphic test generation. These generated MRs will be subsequently passed to the next agent for further validation before proceeding to the test generation.

4) MR Refiner Agent: validates if the system property specified as a test condition from the functional specification is correctly captured and translated into MRs by the *MR Generator Agent*. The validation criteria for each MR take into consideration the factual correctness, category consistency, constraint compliance, causal validity, and testability with respect to the specification of the SUT.

Any missing detail or incorrect information in each MR is verified and refined using refinement rules by the *MR Generator Agent*. In the output schema of the refined MRs, a field named *feedback* provides the information on the refinement performed for each MR, in order to enhance the explainability of the decision, and the field named *dropped* with values as either *'true'/'false'* indicates whether the particular MR should be accepted with refinement or discarded. The repair loop is limited to a configurable number of attempts as defined in the pipeline configuration of the *Coordinator node*. The refined version of the MR in Figure 3 is presented in Figure 4, where the refined part is highlighted in green, and the *feedback* on the refinement performed is highlighted in blue.

5) Test Generator Agent: The *Test Generator Agent* generates a configurable count of metamorphic tests for each MR received for the previous phase. In this step, the simulation parameters of the model, such as the start and end times and the system I/O constraints, are considered. The seed input values are assigned based on the initial conditions in the extraction output. The concrete values for performing the seed to follow-up transformation of input signals are defined in this step using a sampling algorithm. The algorithm uses the initial condition values extracted from the functional specification as the baseline for generating follow-up inputs using the input relation in the *When* part of MR.

The test design also accounts for the rule that any step/ramp change to any input signal should be placed near the start of the simulation time window ($\sim$ 10-25%). The rules also reinforce that any set-point values among the input signals should remain unchanged. Each MR/refined MR from the previous phase guides the generation of one or more tests.

6) Test Validator Agent: The *Test Validator Agent* primarily validates the test inputs and MRs in the test case. It checks the test case against the functional specification and the metamorphic relation patterns used to generate the test inputs. The validation is performed based on repair rules such as data-type and boundary checks for test input values, type of metamorphic relation pattern, and simulation-specific time constraints. In the output schema of validated tests, two fields named *fixed* and *dropped* capture the validation decision, and a summary is provided by the agent in the field *validation_summary*. The decision categorizes a test as any one of the following:

- If the test case is fully valid, keep it unchanged and set fixed=false, dropped=false.
- If the test case can be safely repaired, repair it, apply the smallest possible fix, and set fixed=true, dropped=false.
- If the test case is fundamentally broken or unsafe to repair, set fixed=false, dropped=true.

To summarize, the valid/repaired tests are passed to the test instantiator, which generates the input time series for all the input signals. The test input signals for which the metamorphic transformation is applied capture the evolution of values over time, and the other inputs are represented as constant vectors. These generated input signals are wrapped into an input object and proceed to the test execution phase.

C. Implementation

AgenticMeta is implemented in Python as a layered workflow built around LangGraph [19]. The implementation separates the pipeline into five layers: the orchestration layer, the specification and variable extraction layer, the MR generation and refinement layer, the test generation and validation layer, and the test execution layer. Each layer is integrated with a persistence mechanism that stores its inputs, outputs, intermediate results,

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on September 08,2026 at 00:57:36 UTC from IEEE Xplore. Restrictions apply.

and state, enabling traceability, debugging, and post-run evaluation across the complete workflow. This separation helps with debugging and evaluation, since intermediate artifacts are stored after each phase, the errors can be traced to a specific layer, and individual components can be modified or replaced without affecting the rest of the workflow. The layered design also allows different pipeline settings to be used at different stages, enabling isolated evaluation of each stage, more flexible reasoning early on, and more stable behaviour during execution. Overall, this structure makes AgenticMeta easier to extend and adapt to new tasks.

IV. Evaluation

For evaluation, we use a simplified version of a Lubricating Oil Cooling (LOC) model[1]. The LOC system has a proportional–integral (PI) controlled valve that regulates the lubrication oil temperature at a constant set-point at the engine inlet. The controller aims to keep the lubrication oil temperature at the outlet within the specified boundary values under all operating conditions. The simulation model of the LoC system has been implemented and tested manually by the designers, and in the context of this evaluation, it is used as the ground truth. The case study and tool were deployed locally on a Windows-based PC (Intel Core i9-10900X, 3.70 GHz, 64 GB RAM, 10 GB dedicated graphics card, 2 TB storage). We use FMPy [20], a Python library, to simulate the FMU and execute the tests generated by the workflow.

1) Workflow configuration: We used the following workflow configuration for our experiments. Some of these values were chosen based on empirical evaluations. All agents use OpenAI as the model provider and the GPT-5.4-mini model. Reasoning effort (amount of internal computation the model uses before generating a response) is set to none for the Extraction agent, low for test generation, medium for MR generation and test validator, and high for the MR refinement agent. The Extraction agent uses temperature 0 (temperature is only applicable with *reasoning effort: none*). Each run consists of 5 iterations, with up to 5 MRs per iteration and 5 test cases per MR.

2) Evaluation metrics: We evaluate AgenticMeta using the following metrics: *Requirements coverage* denotes the number of requirements extracted from the specification and mapped into metamorphic test scenarios produced per test session. *Test Case Summary* enumerates generated test cases, their MR assignments, and pass/fail outcomes, providing an indicator of workflow effectiveness. *Runtime statistics* to measure the efficiency of the workflow across different phases and units.

The *Mutation score* is used to measure the quality of metamorphic tests generated by the approach successfully executed (e.g., without any syntax errors) against the LOC. Since we do not have access to the internal specification of the FMU, we imitate possible design and implementation mistakes by applying systematic changes (*mutations*) on the output signals. Test suites that detect these injected faults are more likely to detect real defects as well due to the coupling effect [21]. These mutations are defined based on a set of *mutation operators* that are systematically applied, one at a time, to each output; this will result in a slightly incorrect version (*mutant*) of the FMU. In this work, we use the following mutation operators:

- *Mirror Mutation*: Replaces an output signal with its mirror version from the start time step of the simulation horizon.
- *Crossover Mutation*: Select two output time series of different metamorphic relations for a given testcase and pick a specific crossover site and interchange the values of the time series signals after the crossover site.
- *Polynomial Mutation*: Given a variable x in a time series in the range (x_{min}, x_{max}), we slightly change it by a random amount determined by a polynomial distribution.

The process starts with the execution of the generated test cases on the original version of the FMU. To save on the simulation time, we apply the mutation operators directly on the recorded output time series for each output of the passed test cases. Each application of a mutation operand to an output time series will result in a new mutant. We compare the seed output and the mutated output using the output MRs and assign a pass/fail verdict. Finally, we calculate the *mutation score*, with values between 0 and 1, as the ratio between the number of mutants on which tests failed (killed mutants) and the total number of mutants created, as an indicator of the fault detection capability of our workflow.

3) Results: We evaluated the workflow across 10 independent sessions of the LOC case study. Each run used the same configuration.

a) Answer to RQ1: AgenticMeta successfully executed the complete workflow, including extraction, MR generation, refinement, test generation, validation, instantiation, simulation, and mutation analysis in all ten sessions. The results show that the proposed architecture is feasible for metamorphic testing of FMU-based simulation models. Across all ten sessions, the workflow successfully progressed from requirement extraction to executable metamorphic test generation and execution under the specified configuration. It was also able to perform mutation-based quality assessment on top of the core execution.

b) Answer to RQ2: Across the 10 sessions, the workflow generated an average of 14.90 valid MRs and 55.70 test cases. Compared to a baseline of manually identified MRs for the same case study (depicted as features in [9]), our tool was able to produce, on average, 15 semantically unique MRs per run. Based on manual inspection of the generated MRs across experiments, showed that there was a significant overlap of the MRs, suggesting that the approach can generate stable MR sets across repeated sessions. Despite a consistent MR count, the coverage varies due to the non-deterministic nature of LLMs both in extracting requirements and in selecting distinct subsets when constructing new MRs.

The average requirement coverage was 60.47% (Table II, MR Summary & Req. Coverage). These results indicate that the workflow can repeatedly generate specification-linked MRs and executable metamorphic test cases based on the FMU interface.

[1] https://github.com/Novia-RDI-Seafaring/fmu-opc-hackathon/tree/main/fmus/loc

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on September 08,2026 at 00:57:36 UTC from IEEE Xplore. Restrictions apply.

TABLE I: Runtime statistics across 10 sessions

Sessions	Phase level statistics (s)					Unit level statistics (s)		
	Total Exec.	Extraction	MR Gen	Test Gen	Test Exec	Gen. Time/TC	Gen. Time/MR	Exec. Time/TC
1	771.39	81.74	540.28	110.24	39.14	2.69	45.02	18.82
2	913.54	78.70	617.23	160.21	57.41	2.67	41.14	15.23
3	881.11	81.04	619.95	132.72	47.40	2.77	47.68	18.36
4	1074.30	88.94	773.78	159.77	51.81	2.90	48.36	19.53
5	900.78	79.27	612.57	150.62	58.31	2.55	40.84	15.27
6	951.92	83.76	655.39	157.65	55.12	2.43	40.96	14.64
7	1021.93	80.68	740.52	149.59	51.14	3.05	46.28	20.86
8	943.33	81.05	662.22	147.51	52.55	2.68	47.30	17.15
9	1000.65	97.31	705.96	151.85	45.53	2.81	50.42	18.53
10	908.27	93.94	567.56	184.36	62.41	2.60	31.53	12.79
Average (s)	936.72	84.64	649.55	150.45	52.08	2.72	43.95	17.12

TABLE II: Coverage statistics across 10 sessions

Sessions	MR Summary & Requirement Coverage (%)				Test Summary			Mutation Coverage		
	Gen. MR's	Dropped	Refined	Coverage (%)	Generated	Passed (%)	Failed (%)	Generated	Killed	Score
1	14	2	12	64.71	41	85.37	14.63	104	65	0.63
2	16	1	15	35.29	60	76.67	23.33	176	83	0.47
3	14	1	13	58.82	48	91.67	8.33	162	85	0.52
4	17	1	16	70.59	55	83.64	16.36	174	83	0.48
5	15	0	15	52.94	59	72.88	27.12	162	96	0.59
6	16	0	16	82.35	65	63.08	36.92	94	63	0.67
7	16	0	16	70.59	49	53.06	46.94	138	82	0.59
8	15	1	14	64.71	55	70.91	29.09	158	91	0.58
9	17	3	14	45.83	54	64.81	35.19	78	46	0.59
10	18	0	18	58.82	71	78.87	21.13	126	57	0.45
Average	15.80	0.90	14.90	60.47	55.70	74.10	25.90	137.20	75.10	0.56

The produced MRs were manually checked by domain experts and considered to be meaningful and consistent with the system specification. However, full requirement coverage was not achieved because not all extracted requirements could be converted into valid MRs over the exposed FMU interface. Manual inspection suggested two main causes:

- A few extracted requirements are generic statements about system behaviour and do not define a directly manipulable input-output relation.
- A few requirements described output behaviours that cannot be directly influenced by the available FMU inputs. Such requirements were often removed during the MR refinement stage because the agent reasoned that a valid follow-up output could not be guaranteed with the given inputs.

It was observed that the MRs that were not covered or dropped were mainly those based on indirect, generic, or weakly controllable behaviours. In the LOC case study, MRs based on direct relations between controllable inputs and observable outputs were more stable, while MRs requiring indirect influence over variables were more likely to be refined or removed. This indicates that the approach is most effective when the specification states explicit causal links between input perturbations and expected output behaviour.

c) Answer to RQ3: The test case pass rate across simulation sessions averaged 74.10% (see Table II, Test Summary). This indicates that most generated tests were semantically aligned with the generated MRs. As the implementation of the LOC is considered to be correct, the failing tests in this case are considered false positives and a limitation of our workflow, which we will address in future work. Further investigation showed that several failures occurred when the output settled outside the specified tolerance range or around a value different from the setpoint values. These observations indicate that future versions of the workflow should include stronger validation of temporal operators and tolerance values.

The average mutation score across the ten sessions was 0.56, corresponding to 56% of the generated mutants killed. (see Table II, Mutation Coverage). These results indicate that the majority of mutations are detected by the evaluator, but they do not exhaustively cover the model behaviour.

d) Answer to RQ4: The average test-generation time is 2.72s per test case, the average MR-generation and refinement time is 43.95s per accepted MR, & the average execution time per test case is 17.12s (see Table I, Unit level stats.). These results show that the overall time taken for the complete loop from extraction to execution is very low per test case. The compact test-case schema generated by the test generator agent is effective in keeping the test generation phase relatively fast.

The main efficiency bottleneck is the LLM-based MR generation and refinement. The refinement phase remains comparatively expensive due to the 'high' reasoning effort of *MR Refiner Agent*, and also it checks generated MRs against both structured extraction output and the original functional specification, making it a resource-intensive process. The result

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on September 08,2026 at 00:57:36 UTC from IEEE Xplore. Restrictions apply.

suggests that future optimization should focus on reducing unnecessary MR refinement calls, prioritizing uncovered requirements more selectively, caching stable extraction artifacts, and using less expensive models for low-risk validation steps.

V. Threats to Validity

The evaluation is conducted on a single FMU-based case study, the LOC system. Although LOC is representative of dynamic simulation models with continuous inputs and outputs, the findings may not generalize to models with substantially different control logic, interface structures, or operating characteristics. Nevertheless, the results show that such a workflow is feasible for metamorphic test generation.

Second, the mutation operators defined in this work are based on hypothetical failure models, such as logic errors or sensor failures, and are not directly linked to the real simulation faults from historical data. Therefore, the mutation results should be interpreted cautiously. Because mutations are applied to recorded output time series rather than to the FMU implementation, the score reflects the sensitivity of the MRs to selected output deviations, not the full fault-detection capability of the workflow against implementation defects.

Third, the workflow depends on the quality of the functional specification. As the implementation uses document conversion tools and evidence-preserving extraction prompts, the generated MRs are directly constrained on the correctness and completeness of the specification document.

Last, the choice of LLM model influences the quality of the approach. In this study, GPT-5.4-mini was used, but alternative or future models may produce different results depending on their reasoning capability and output quality.

VI. Conclusions and Future Work

This paper proposes AgenticMeta, a multi-agent workflow for specification-based metamorphic testing of FMU-based simulation models. The approach used functional and interface specifications to extract test-relevant requirements, generate and refine MRs, produce executable metamorphic test cases, and evaluate test outcomes using metamorphic oracles and mutation analysis. The feasibility evaluation was performed on the LOC system, which shows that the workflow can stably execute the complete testing across repeated sessions. AgenticMeta generated specification-linked MRs and executable test cases, achieved an average requirement coverage of 60.47%, an average test-case pass rate of 74.10%, and an average mutation score of 56%. These results indicate that the approach is feasible for supporting automated metamorphic testing of FMU-based simulations. Requirement coverage is constrained by the quality and testability of the specification.

Future work will extend the workflow by transforming the deterministic *Coordinator node* into an LLM-based agent capable of making adaptive decisions during execution. Further improvements will focus on designing mutation operators that are more closely aligned with the semantics of the evaluated MRs making mutation-based assessment more robust for FMU systems. We will also explore the use of different LLM models, inference settings, and tool integrations to assess how model choice and toolchain design influence MR-generation quality, workflow stability, and overall testing effectiveness.

Acknowledgments

This work was funded by the Finnish Ministry of Education and Culture's Doctoral Education Pilot under Decision No. VN/3137/2024-OKM-6 (The Finnish Doctoral Program Network in AI, AI-DOC) and Business Finland via the Virtual Sea Trial project (VST), under grant 7187/31/2023.

References

[1] J. Cederbladh et al. Early validation and verification of system behaviour in model-based systems engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology*, 33(3), 2024.

[2] T. Blochwitz et al. Functional mockup interface 2.0: The standard for tool independent exchange of simulation models. In *9th international modelica conference*, pp. 173–184. The Modelica Association, 2012.

[3] T. Y. Chen et al. Metamorphic testing: a new approach for generating next test cases. *arXiv preprint arXiv:2002.12543*, 2020.

[4] H. Liu et al. A new method for constructing metamorphic relations. In *12th International Conference on Quality Software*. IEEE, 2012.

[5] S. Segura and Z. Q. Zhou. Metamorphic testing 20 years later: A hands-on introduction. In *Proceedings of the 40th International Conference on Software Engineering: Companion Proceeedings*, pp. 538–539, 2018.

[6] A. Núñez and R. M. Hierons. A methodology for validating cloud models using metamorphic testing. *annals of telecommunications-annales des télécommunications*, 70(3):127–135, 2015.

[7] M. Lindvall et al. Metamorphic model-based testing of autonomous systems. In *2017 IEEE/ACM 2nd International Workshop on Metamorphic Testing (MET)*, pp. 35–41. IEEE, 2017.

[8] M. Olsen and M. Raunak. Increasing validity of simulation models through metamorphic testing. *IEEE Trans. on Reliability*, 68(1), 2018.

[9] G. Sudheerbabu et al. Validation of dynamic simulation models using metamorphic testing and given-when-then patterns. In *Modelica Conferences*, pp. 139–146, 2025.

[10] S. Segura et al. A survey on metamorphic testing. *IEEE Transactions on Software Engineering*, 42(9):805–824, 2016.

[11] T. Y. Chen et al. Metamorphic testing: A review of challenges and opportunities. *ACM Computing Surveys (CSUR)*, 51(1):1–27, 2018.

[12] Q. H. Luu et al. Can ChatGPT Advance Software Testing Intelligence? An Experience Report on Metamorphic Testing. *arXiv:2310.19204*, 2023.

[13] Y. Zhang et al. Automated Metamorphic-Relation Generation with ChatGPT. In *Proceedings of the 47th IEEE Annual Computers, Software, and Applications Conference (COMPSAC)*, pp. 1–6. IEEE, 2023.

[14] S. Y. Shin et al. Towards generating executable metamorphic relations using large language models. In *Intl. Conf. on the Quality of Information and Communications Technology*, pp. 126–141. Springer, 2024.

[15] M. Wynne and A. Hellesoy. *"The cucumber book: behaviour-driven development for testers and developers"*. Pragmatic Bookshelf, 2012.

[16] S. Khan et al. Multi-Agent LLM-based Metamorphic Testing for REST APIs, 2026.

[17] P. C. Cañizares et al. Towards metamorphic testing with LLM-based workflows: Metamorphic relation inference and follow-up test case generation. *Information and Software Technology*, 196:108150, 2026.

[18] L. Liang et al. AutoMT: A Multi-Agent LLM Framework for Automated Metamorphic Testing of Autonomous Driving Systems. *arXiv preprint arXiv:2510.19438v1*, 2025.

[19] LangChain. LangGraph. https://reference.langchain.com/python/langgraph/overview, 2024. Accessed: 2026-04-30.

[20] DassaultSystémes. FMPy. https://fmpy.readthedocs.io/en/latest/, 2017. Accessed: 2026-04-21.

[21] A. J. Offutt. Investigations of the software testing coupling effect. *ACM Trans. Softw. Eng. Methodol.*, 1(1):5–20, January 1992.

Authorized licensed use limited to: BEIHANG UNIVERSITY. Downloaded on September 08,2026 at 00:57:36 UTC from IEEE Xplore. Restrictions apply.